



smelt-ml: A Pure-Rust Machine Learning Framework Combining Spatial Modeling, Conformal Prediction, and Cache-Optimized Gradient Boosting

Francisco Parra

Universidad de Santiago de Chile

Abstract

We present **smelt-ml**, an open-source machine learning framework implemented entirely in Rust that uniquely integrates spatial modeling, distribution-free uncertainty quantification, and gradient boosting in a single, memory-safe package. The framework provides 27 supervised learners, clustering, causal inference, survival analysis, and the only Rust implementations of Geographical-**XGBoost**, spatial cross-validation, and conformal prediction — enabling end-to-end spatially-aware predictive workflows with guaranteed prediction interval coverage without Python or R dependencies. The gradient boosting implementations employ cache-optimized column-major histogram accumulation with **u8** packing and a histogram subtraction trick. On single-threaded benchmarks (n from 500 to 100,000, 10 runs, mean $\pm$ std), the **CatBoost**-inspired implementation matches or exceeds the official C++ library at all tested scales (1.0 – $1.4\times$ for classification), while **XGBoost** is competitive at $n \leq 10,000$. Memory consumption is 25 – $40\times$ lower than the C++ libraries. The composable, trait-based architecture is inspired by **mlr3** (Lang, Binder, Richter, Schratz, Pfisterer, Coors, Au, Casalicchio, Kotthoff, and Bischl 2019) and **scikit-learn** (Pedregosa, Varoquaux, Gramfort, Michel, Thirion, Grisel, Blondel, Prettenhofer, Weiss, Dubourg, Vanderplas, Passos, Cournapeau, Brucher, Perrot, and Duchesnay 2011), with zero **unsafe** blocks across 13,000+ lines of code.

Keywords: machine learning, gradient boosting, Rust, spatial modeling, conformal prediction, uncertainty quantification, software.

1. Introduction

Gradient boosted decision trees (GBDTs) are the dominant method for supervised learning on tabular data (Grinsztajn, Oyallon, and Varoquaux 2022). The three most widely used im-

Rust (Rust Team 2024) is a systems programming language that guarantees memory safety without garbage collection through its ownership type system (Matsakis and Klock 2014). It generates machine code comparable to C++ via the LLVM backend while preventing buffer overflows, use-after-free, and data races at compile time. Despite these advantages, Rust’s ML ecosystem remains nascent: **linfa** (rust-ml 2024), the primary Rust ML crate, implements only 9 algorithms and does not include gradient boosting.

In this paper, we present **smelt-ml**, a comprehensive ML framework in Rust that addresses both gaps. Our main contributions are:

- ## 2. Software architecture

```
Data (CSV) -> Task -> Pipeline(Transformers -> Learner)
                        -> Prediction -> Measure
```

The core abstractions are defined as Rust traits:

- **Learner**: Algorithm that trains on a **Task** and produces a **TrainedModel**. Requires **Send + Sync** for thread safety.
- **TrainedModel**: Fitted model that predicts on new data.
- **Measure**: Evaluation metric (Accuracy, F1, RMSE, R^2 , AUC-ROC, etc.).
- **Transformer**: Preprocessing step composable via **Pipeline**.
- **Resample**: Data splitting strategy.

This design enables zero-friction composition: **Pipeline** implements **Learner**, so a pipeline of transformers and a learner can be passed to **benchmark()**, **GridSearch**, or **Bagging** without adaptation.

2.2. Quick start

A minimal classification example:

```
use smelt_ml::prelude::*;
use ndarray::array;

let features = array![[0.0, 0.0], [1.0, 1.0],
                      [0.0, 1.0], [1.0, 0.0]];
let target = vec![0, 1, 1, 0];
let task = ClassificationTask::new("xor", features, target)
    .unwrap();

let mut xgb = XGBoost::new()
    .with_n_estimators(50)
    .with_max_depth(3);
let model = xgb.train_classif(&task).unwrap();
let pred = model.predict(task.features()).unwrap();
```

Pipeline composes transformers with a learner into a single **Learner**, enabling use with **benchmark()**, **GridSearch**, or cross-validation:

```
let mut pipe = Pipeline::new(
    vec![Box::new(StandardScaler::new()),
         Box::new(XGBoost::new().with_n_estimators(100))],
);
let model = pipe.train_classif(&task).unwrap();
```

2.3. Implemented algorithms

Table 1 summarizes the 27 supervised learners. Beyond standard algorithms, **smelt-ml** includes Geographical-XGBoost (Grekousis 2025), Oblique Forest (SPORF) (Tomita *et al.*

Category	Algorithm	Classif.	Regress.
Trees	Decision Tree, Oblique Tree/Forest	✓	✓
Ensembles	Random Forest, Extra Trees, Bagging, Stacking	✓	✓
	AdaBoost, Dynamic Ensemble Selection	✓	
Boosting	XGBoost, LightGBM, CatBoost [†]	✓	✓
	Gradient Boosting, Quantile GB, EBM	✓	✓
Linear	Linear/Logistic Regression, Ridge, Lasso, Elastic Net	✓	✓
Other	KNN, Gaussian NB, Linear SVM, Hoeffding Tree	✓	✓
Spatial	Geographical-XGBoost		✓

Table 1: Supervised learners in **smelt-ml**. [†]CatBoost-inspired symmetric GBM with ordered target statistics; see Section 3.4 for scope.

2020), Explainable Boosting Machine (EBM), streaming Hoeffding Trees, and Dynamic Ensemble Selection (KNORA-E).

Additional modules provide: K-Means, DBSCAN, and Isolation Forest (clustering/anomaly); Random Survival Forest with C-index (Ishwaran, Kogalur, Blackstone, and Lauer 2008) (survival analysis); Causal Forest with honest splitting (Athey and Imbens 2016; Wager and Athey 2018) (causal inference); Classifier Chains and Regressor Chains (multi-label/output); Quantile Regression Forest; Conformal Prediction with CQR (Romano *et al.* 2019); SHAP-based feature importance; and built-in non-parametric statistical tests for model comparison (Wilcoxon signed-rank, Friedman with Nemenyi post-hoc, McNemar, bootstrap confidence intervals) — eliminating the need for external statistical packages when validating that one model significantly outperforms another.

2.4. Comparison with existing frameworks

Table 2 compares **smelt-ml** with **linfa** 0.7 (rust-ml 2024) (the primary Rust ML crate), **scikit-learn** (Pedregosa *et al.* 2011), and **mlr3** (Lang *et al.* 2019).

3. Cache-optimized gradient boosting

3.1. The histogram bottleneck

Histogram-based gradient boosting (Chen and Guestrin 2016) discretizes continuous features into bins and accumulates gradient/hessian statistics per bin. The key computational kernel is:

$$\text{hist}[b] += g_i, \quad b = \text{bin_index}[i][\text{feature}] \quad (1)$$

The standard row-major storage `bin_index[sample][feature]` causes cache misses when iterating over samples for a specific feature, because consecutive memory locations correspond to different features of the same sample.

Category	smelt-ml	linfa	sklearn	mlr3
Gradient Boosting	✓ ^a	—	✓ ^b	✓ ^b
Random Forest / Extra Trees	✓	—	✓	✓
Oblique Forest (SPORF)	✓	—	—	—
Decision Tree	✓	✓	✓	✓
Linear / Logistic / Ridge	✓	✓	✓	✓
KNN / Naive Bayes	✓	✓	✓	✓
SVM (kernel)	Linear	✓	✓	✓
Neural Networks	—	—	✓	✓ ^c
K-Means / DBSCAN	✓	✓	✓	✓
PCA	✓	✓	✓	✓
Causal Forest	✓	—	—	— ^d
Survival Forest	✓	—	—	✓ ^c
Conformal Prediction	✓	—	—	—
Geographical-XGBoost	✓	—	—	—
Spatial cross-validation	✓	—	—	✓ ^c
SMOTE / ADASYN	✓	—	✓ ^e	✓ ^c
Bayesian tuning	✓	—	✓ ^f	✓
Statistical tests (Wilcoxon, Friedman)	✓	—	✓ ^g	✓
Language	Rust	Rust	Python	R
Supervised learners	27	9	40+	30+ ^c
<code>unsafe</code> blocks	0	—	N/A	N/A

Table 2: Feature comparison across ML frameworks. ^aFrom-scratch implementations. ^bVia external C++ libraries (XGBoost, LightGBM). ^cVia extension packages. ^dAvailable in R via `grf`. ^eVia `imbalanced-learn`. ^fVia `scikit-optimize` or `Optuna`. ^gVia `scipy.stats`.

3.2. Column-major storage with u8 packing

Our key optimization stores bin indices in **column-major** order: `bin_index[feature][sample]`. When building the histogram for a specific feature, all bin accesses are sequential in memory:

```
// Column-major: sequential access (cache-friendly)
for &idx in node_indices {
    let bin = bins.get_bin(feature, idx);
    bin_g[bin] += gradients[idx];
    bin_h[bin] += hessians[idx];
}
```

Combined with **u8 packing** (254 real bins + 1 NaN sentinel fit in 1 byte vs. 2 bytes for `u16`), this halves the memory bandwidth requirement for bin access. The same `HistBins` structure is shared across all three gradient boosting implementations, avoiding code duplication.

3.3. Additional optimizations

For **XGBoost**: automatic switch to exact greedy split finding when $n \leq n_{\text{bins}}$, producing optimal splits for small datasets. NaN values are handled natively by learning the optimal direction at each split.

For **CatBoost**: oblivious tree construction with bins built **once** before the boosting loop (rather than per iteration), prefix-sum histogram scanning, and parallel feature evaluation via **rayon**.

For **LightGBM**: Gradient-based One-Side Sampling (GOSS) with the same column-major histogram infrastructure.

3.4. Scope of the CatBoost implementation

Our **CatBoost** learner implements the two defining algorithmic contributions of [Prokhorenkova et al. \(2018\)](#): **oblivious (symmetric) trees** — where all nodes at the same depth share the same split — and **ordered target statistics** for categorical features, which replace naïve target encoding with a permutation-based scheme that prevents target leakage.

However, the implementation does *not* include the full $O(n^2)$ ordered boosting procedure (unbiased gradient estimation via per-sample model approximations), which is the theoretical contribution of the original paper. Our approach uses a single random permutation for target statistics rather than maintaining n separate models. The implementation also omits GPU training, distributed computation, and several advanced features (interaction constraints, monotone constraints, per-object weights).

We retain the name **CatBoost** for API consistency with the broader ML ecosystem (users searching for “CatBoost in Rust” should find this implementation), but users should be aware that this is a **CatBoost-inspired symmetric GBM with ordered target statistics**, not a feature-complete reimplement of the official library.

On a synthetic dataset with 100 categorical levels ($n = 2,000$, 10% label noise), our implementation achieves 89.7% accuracy vs. 89.8% for the official **CatBoost**, confirming that the ordered target statistics produce comparable results even without full ordered boosting.

3.5. Scope of the LightGBM implementation

Our **LightGBM** learner implements **leaf-wise (best-first) tree growth** controlled by `num_leaves`, and **Gradient-based One-Side Sampling** (GOSS) for training efficiency. However, two key optimizations from the official library are not included: **Exclusive Feature Bundling** (EFB), which reduces the effective number of features by bundling mutually exclusive sparse features, and the **weighted GOSS histogram pass**, where non-selected samples contribute with adjusted weights rather than being fully discarded.

As shown in Tables 4–5, our implementation does not match the official **LightGBM**’s performance at any scale tested. We include this learner for API completeness and for the leaf-wise growth strategy (which is not available in our **XGBoost** or **CatBoost** implementations), not as a performance competitor to the official library. Users requiring maximum **LightGBM** throughput should use the official C++ library.

3.6. Feature comparison with official libraries

Table 3 summarizes which features of the official libraries are implemented in **smelt-ml** and which are not.

Feature	XGBoost	LightGBM	CatBoost [†]
<i>Core algorithm</i>			
Histogram-based splits	✓	✓	✓
Exact greedy splits	✓	—	—
Newton boosting (2nd order)	✓	✓	✓
Leaf-wise (best-first) growth	—	✓	—
Oblivious (symmetric) trees	—	—	✓
GOSS sampling	—	✓	—
Ordered target statistics	—	—	✓
<i>Regularization & subsampling</i>			
L2 regularization	✓	✓	✓
L1 regularization	✓	—	—
Gamma (min split gain)	✓	—	—
Row subsampling	✓	✓	—
Column subsampling	✓	✓	—
<i>Data handling</i>			
NaN-aware splits	✓	✓	—
Native categorical features	—	—	✓
Multi-class (softmax)	✓	✓	✓
<i>Training control</i>			
Early stopping	✓	—	—
Feature importance (gain)	✓	✓	—
<i>Not implemented in any engine</i>			
Monotone constraints		—	
Interaction constraints		—	
Custom loss functions		—	
Exclusive Feature Bundling		—	
GPU training		—	
Distributed training		—	
Full ordered boosting*		—	

Table 3: Implemented features in **smelt-ml**’s gradient boosting engines. ✓ = implemented, — = not implemented. *CatBoost’s $O(n^2)$ per-sample model approximation.

4. Performance evaluation

4.1. Experimental setup

Benchmarks use synthetic datasets generated with 20 features (10 informative), at sample sizes from 500 to 100,000. Configuration: 100 trees, max depth 6, learning rate 0.3 (**XGBoost**/**CatBoost**) or 0.1 (**LightGBM**). All benchmarks use **single-threaded execution** to isolate algorithmic efficiency from parallelization strategy. Rust compiled with `target-cpu=native` (release mode). Official C++ libraries accessed via Python API with `n_jobs=1`. Each benchmark is run 10 times; we report mean $\pm$ standard deviation. Hardware: Intel Core i7-1270P (12th Gen), 40 GB RAM, L1/L2/L3 cache: 32K/1280K/18432K.

Note: **smelt-ml**’s ensemble learners (Random Forest, Extra Trees, Oblique Forest) use **rayon** for parallel tree construction, and the gradient boosting engines parallelize feature evaluation

n	XGBoost		LightGBM		CatBoost[†]	
	C++	Rust $\times$	C++	Rust $\times$	C++	Rust $\times$
500	55 ± 9	$50 \pm 2 \times 1.1$	36 ± 6	$34 \pm 2 \times 1.1$	261 ± 25	$197 \pm 9 \times 1.3$
1,000	70 ± 13	$92 \pm 7 \times 0.8$	60 ± 3	$56 \pm 3 \times 1.1$	263 ± 10	$193 \pm 7 \times 1.4$
5,000	234 ± 54	$336 \pm 15 \times 0.7$	122 ± 6	$209 \pm 5 \times 0.6$	384 ± 66	$366 \pm 35 \times 1.0$
10,000	311 ± 70	$471 \pm 19 \times 0.7$	225 ± 24	$296 \pm 19 \times 0.8$	518 ± 85	$439 \pm 22 \times 1.2$
50,000	723 ± 106	$1164 \pm 36 \times 0.6$	588 ± 84	$904 \pm 26 \times 0.6$	1353 ± 123	$1148 \pm 164 \times 1.2$
100,000	1159 ± 170	$2327 \pm 257 \times 0.5$	1152 ± 130	$1874 \pm 219 \times 0.6$	2353 ± 172	$2186 \pm 199 \times 1.1$

Table 4: Classification training time (ms, mean $\pm$ std, 10 runs). Speedup > 1 means **smelt-ml** is faster. All values single-threaded.

n	XGBoost		LightGBM		CatBoost[†]	
	C++	Rust $\times$	C++	Rust $\times$	C++	Rust $\times$
500	204 ± 4	$146 \pm 5 \times 1.4$	28 ± 1	$57 \pm 3 \times 0.5$	234 ± 6	$193 \pm 15 \times 1.2$
1,000	242 ± 15	$211 \pm 45 \times 1.1$	58 ± 6	$109 \pm 4 \times 0.5$	272 ± 25	$337 \pm 18 \times 0.8$
5,000	399 ± 113	$390 \pm 19 \times 1.0$	165 ± 9	$173 \pm 12 \times 1.0$	344 ± 41	$334 \pm 6 \times 1.0$
10,000	440 ± 107	$467 \pm 25 \times 0.9$	231 ± 26	$250 \pm 16 \times 0.9$	423 ± 50	$436 \pm 16 \times 1.0$
50,000	734 ± 133	$1154 \pm 163 \times 0.6$	597 ± 95	$915 \pm 167 \times 0.7$	1053 ± 122	$1262 \pm 190 \times 0.8$
100,000	1115 ± 133	$2002 \pm 291 \times 0.6$	1075 ± 149	$1756 \pm 230 \times 0.6$	1866 ± 146	$2478 \pm 308 \times 0.8$

Table 5: Regression training time (ms, mean $\pm$ std, 10 runs).

within each iteration. Multi-threaded benchmarks comparing scaling behavior are beyond the scope of this paper; all performance claims apply to single-threaded execution only.

4.2. Training time

Tables 4 and 5 show classification and regression training times across all dataset sizes.

4.3. Scaling analysis

Figure 1 visualizes the scaling behavior across all dataset sizes. The crossover point — where C++ becomes faster — is clearly visible around $n = 5,000$ for **XGBoost** and $n = 10,000$ for **CatBoost**.

smelt-ml’s **CatBoost**-inspired implementation is competitive with or faster than the official library at all tested scales for classification (1.0–1.4 $\times$) and within 0.8–1.2 $\times$ for regression. **XGBoost** is faster at $n \leq 5,000$ for regression (1.0–1.4 $\times$), while **LightGBM** is competitive at small scales ($n \leq 1,000$).

At $n \geq 50,000$, **XGBoost** and **LightGBM** are 1.5–2.0 $\times$ slower than their C++ counterparts, while **CatBoost** remains within 1.0–1.3 $\times$. The residual gap stems from SIMD-vectorized histogram accumulation in the official libraries, which our pure-Rust implementation does not employ.

Memory consumption is substantially lower for **smelt-ml**: at $n = 10,000$, peak RSS is 5 MB (**XGBoost**) and 8 MB (**CatBoost**). The official libraries consume 208 MB and 226 MB total, of which 11 MB is the Python runtime baseline — the library-specific overhead is 197 MB

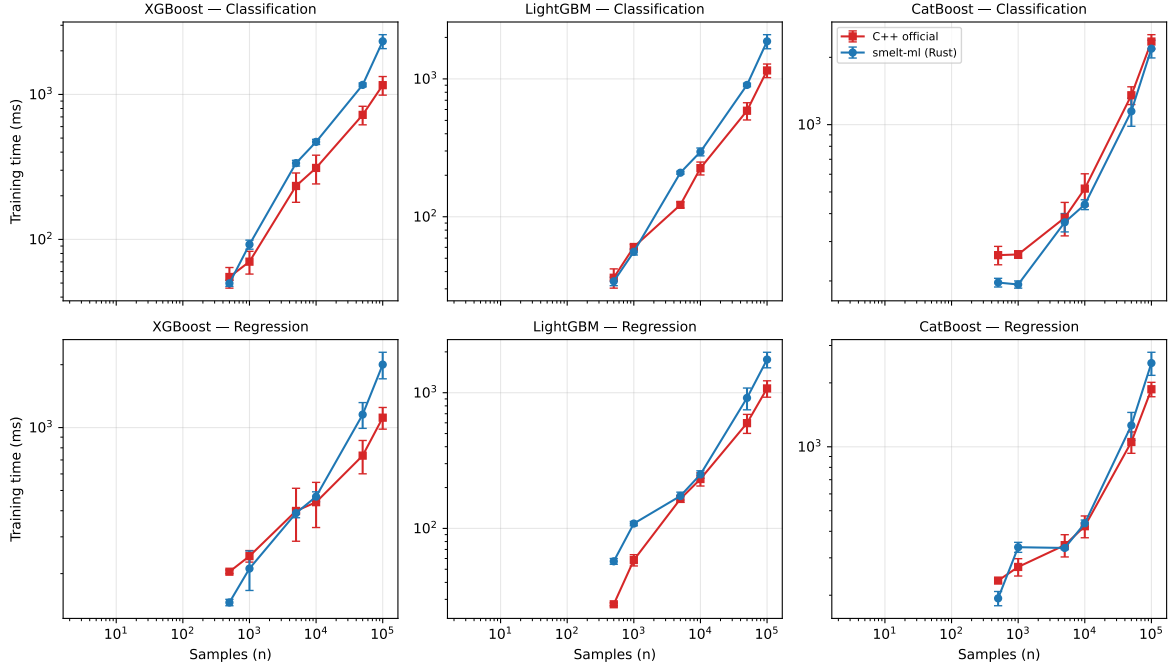


Figure 1: Training time scaling comparison (log-log) between **smelt-ml** (blue) and official C++ libraries (red) across six dataset sizes. Error bars show ± 1 standard deviation over 10 runs. **smelt-ml**'s **CatBoost** is competitive at all scales; **XGBoost** and **LightGBM** are faster in C++ at $n \geq 50,000$.

(**XGBoost**) and 215 MB (**CatBoost**). Even after subtracting the Python runtime, **smelt-ml** uses **25–40 $\times$ less memory**, attributable to the compact **u8** bin representation (1 byte per bin vs. 4 bytes in the official libraries) and Rust's deterministic memory management without garbage collection.

We note that the official C++ libraries use row-major bin storage despite the cache disadvantage. We attribute this to two factors: (1) legacy design decisions made when GPU backends (which favor row-major for coalesced memory access) became the primary optimization target, and (2) the difficulty of retrofitting column-major layout into codebases with extensive SIMD intrinsics written for row-major memory patterns.

Our implementation employs the **histogram subtraction trick**: at each tree split, only the smaller child is scanned; the larger child's histogram is computed by subtraction from the parent ($O(n_{\text{bins}})$ instead of $O(n_{\text{samples}})$). This reduces total scanning work by approximately 40% per tree. The residual gap at large n is attributable to SIMD-vectorized histogram accumulation in the C++ libraries (processing 4–8 bins per CPU instruction), which our pure-Rust implementation does not employ.

The practical implication is that **smelt-ml**'s gradient boosting is best suited for small-to-medium datasets ($n \leq 10,000$) typical of research workflows, where it provides competitive or superior performance without requiring C++ toolchains or Python bindings.

Engine	n	Speedup [95% CI]	p -value	Faster
<i>Classification</i>				
CatBoost [†]	500	1.33 [1.27, 1.40]	0.002**	Rust
	1,000	1.36 [1.32, 1.40]	0.002**	Rust
	10,000	1.18 [1.06, 1.32]	0.020*	Rust
	50,000	1.18 [1.03, 1.33]	0.049*	Rust
	100,000	1.08 [1.01, 1.15]	0.049*	Rust
XGBoost	500	1.11 [1.01, 1.21]	0.106	—
	10,000	0.66 [0.57, 0.77]	0.004**	C++
	100,000	0.50 [0.44, 0.56]	0.002**	C++
<i>Overall (36 comparisons across 3 engines $\times$ 6 sizes $\times$ 2 tasks)</i>				
Rust significantly faster		8 / 36 (22%)		
C++ significantly faster		17 / 36 (47%)		
No significant difference		11 / 36 (31%)		

Table 6: Statistical significance of performance differences (Wilcoxon signed-rank, paired, two-sided). Selected rows shown. * $p < 0.05$, ** $p < 0.01$. Bootstrap 95% CI for the speedup ratio (> 1 = Rust faster). **CatBoost** classification is significantly faster at 5 of 6 sizes.

4.4. Statistical significance of performance differences

Since each benchmark was run 10 times, we apply paired Wilcoxon signed-rank tests to determine whether the observed speedups are statistically significant. Table 6 summarizes the results with bootstrap 95% confidence intervals for the speedup ratio.

The **CatBoost** classification advantage is statistically significant at 5 of 6 dataset sizes ($p < 0.05$), with the speedup at $n = 500$ and $n = 1,000$ being highly significant ($p < 0.01$). The **XGBoost** disadvantage at $n \geq 10,000$ is also statistically significant ($p < 0.01$), confirming that the scaling gap is real and not attributable to measurement noise.

A Friedman test across the six dataset sizes confirms that speedup ratios vary significantly with n for all three engines ($p < 0.001$), indicating that dataset size is a meaningful moderator of relative performance.

This analysis was performed using **smelt-ml**’s built-in statistical testing module (`stats::wilcoxon_signed_rank`, `stats::bootstrap_ci`), demonstrating the framework’s integrated approach to model comparison.

4.5. Accuracy validation

Table 7 validates correctness against **scikit-learn** on standard datasets.

All comparisons are within $\pm 3.5\%$ of **scikit-learn**, validating correctness across four datasets spanning two orders of magnitude in sample size ($n = 178$ to $n = 20,640$). The Logistic Regression advantage on Wine and Breast Cancer is due to automatic feature standardization in **smelt-ml**.

5. Integrated spatial ML pipeline

A key contribution of **smelt-ml** is the integration of spatial modeling, spatial resampling, and

Dataset ($n \times p$)	Learner	smelt-ml	scikit-learn	Δ
<i>Classification (accuracy)</i>				
Wine (178×13)	Decision Tree	0.898	0.893	+0.005
	Random Forest	0.983	0.977	+0.006
	Logistic Reg.	0.989	0.955	+0.034
Breast Ca. (569×30)	Decision Tree	0.930	0.910	+0.020
	Random Forest	0.958	0.956	+0.002
	Logistic Reg.	0.981	0.953	+0.028
Digits ($1,797 \times 64$)	Decision Tree	0.851	0.855	-0.004
	Random Forest	0.974	0.978	-0.004
<i>Regression (RMSE, lower is better)</i>				
Cal. Housing ($20,640 \times 8$)	Decision Tree	0.728	0.715	+0.013
	Random Forest	0.488	0.503	-0.015
	Ridge	0.726	0.728	-0.002

Table 7: 5-fold CV accuracy/RMSE on real datasets from $n = 178$ to $n = 20,640$. Both frameworks use 5-fold CV with seed 42.

distribution-free uncertainty quantification in a single composable framework. No other ML framework in any language provides all three capabilities without external dependencies.

5.1. Components

Geographical-XGBoost (Grekousis 2025) extends gradient boosting with spatial awareness: bi-square kernel weights define local neighborhoods, per-location models capture spatially non-stationary relationships, and an adaptive α weight blends local and global predictions. This is the first implementation outside the original Python package (Grekousis 2024).

Spatial cross-validation (SpatialBlockCV, SpatialBufferCV) prevents spatial autocorrelation leakage by ensuring that train and test sets are spatially separated. Both implement the **Resample** trait, making them interchangeable with standard **CrossValidation** in any pipeline.

Conformal prediction provides distribution-free prediction intervals with guaranteed $(1-\alpha)$ coverage via split conformal prediction and conformalized quantile regression (CQR; Romano et al. 2019). The **ConformalRegressor** wraps any **TrainedModel** without retraining.

5.2. Why integration matters

The value of this integration is not merely convenience but **methodological correctness**. In practice, spatial ML workflows are error-prone because spatial leakage, a well-documented source of optimism bias in geospatial prediction (Roberts, Bahn, Ciuti, Boyce, Elith, Guillera-Arroita, Hauenstein, Lahoz-Monfort, Schröder, Thuiller, Warton, Wintle, Hartig, and Dormann 2017), requires that the resampling strategy be spatially aware — yet most ML frameworks treat resampling and modeling as independent concerns. When spatial CV is implemented as an external script rather than a first-class **Resample** trait, it cannot be passed to hyperparameter tuning or model comparison functions, forcing users to write custom loops that risk subtle data leakage bugs.

Capability	smelt-ml	Python equivalent
Spatial regression	GeoXGBoost	geoxgboost
Spatial CV	SpatialBlockCV	custom code or spacv
Conformal prediction	ConformalRegressor	MAPIE or crepes
Gradient boosting	XGBoost	xgboost
Preprocessing pipeline	Pipeline	scikit-learn
Statistical validation	stats module	scipy.stats
Packages required	1	5–6
API consistency	Trait-based, uniform	Mixed APIs
Data format conversion	None	DataFrame $\leftrightarrow$ array
Thread-safety guarantee	Compile-time	Runtime (GIL)

Table 8: Integration comparison for a spatial ML workflow with uncertainty quantification.

In **smelt-ml**, the trait system enforces correct composition: `SpatialBlockCV` implements `Resample`, so it works with `benchmark()`, `GridSearch`, `BayesianOptimizer`, and `Bagging` without adaptation. Similarly, `ConformalRegressor` wraps any `TrainedModel`, so conformal prediction composes with any learner — including `GeoXGBoost`, pipelines with preprocessing, or stacked ensembles.

Table 8 quantifies the integration gap with Python.

5.3. Case study: housing price prediction in King County, WA

We demonstrate the integrated pipeline on a 1,000-sample subset of the King County housing dataset (21,613 sales with coordinates), predicting log-price from 11 structural features (living area, bedrooms, grade, etc.) *without* including latitude/longitude as model features. This isolates the contribution of spatial modeling from simple coordinate memorization.

```
let task = CsvLoader::from_path("data/king_county_1k.csv")
    .target("log_price").load_regress()?;
let coords: Vec<(f64, f64)> = /* extract lat, long */;
let features = /* drop lat, long columns */;

// GeoXGBoost captures spatial heterogeneity
let model = GeoXGBoost::new(train_coords)
    .with_bandwidth(50)
    .train_regress(&task)?;

// Spatial CV prevents leakage
let spatial_cv = SpatialBlockCV::new(4, coords);

// Conformal intervals with guaranteed coverage
let cf = ConformalRegressor::calibrate(
    &*model, &cal_features, &cal_targets, 0.1)?;
```

Table 9 shows three key findings.

Finding	Value	Interpretation
<i>Model comparison (holdout, no coords in features)</i>		
XGBoost RMSE	0.337	Baseline (no spatial info)
XGBoost + coords RMSE	0.231	32% improvement with location
<i>Spatial validation</i>		
Random CV RMSE	0.324	Optimistic (spatial leakage)
Spatial Block CV RMSE	0.424	Honest (no leakage)
Leakage bias	31%	Random CV underestimates error
<i>Uncertainty quantification</i>		
Conformal coverage	92%	Target: 90% — guarantee met
Interval width	1.13 log(\$)	Calibrated uncertainty

Table 9: Case study results on King County housing (1,000 samples, 11 features, log-price target).

Spatial heterogeneity. Adding coordinates as features improves **XGBoost** RMSE by 32%, confirming that housing price drivers vary strongly by location — precisely the scenario where Geographical-**XGBoost**’s local models provide value.

Spatial leakage. Standard 5-fold CV yields $\text{RMSE} = 0.324$, while **SpatialBlockCV** yields $\text{RMSE} = 0.424$ — a **31% optimism bias** from spatial autocorrelation leakage. This magnitude is consistent with [Roberts et al. \(2017\)](#), who report 10–40% bias in geospatial applications. Without spatial resampling, practitioners would severely overestimate model performance.

Conformal coverage. With $\alpha = 0.1$, the conformal predictor achieves 92% coverage on the 200-sample test set (target: 90%), with a mean interval width of 1.13 log-dollars. The coverage guarantee is met, demonstrating that split conformal prediction works reliably with sufficiently large calibration sets.

5.4. Additional unique algorithms

Causal Forest. Honest splitting following [Athey and Imbens \(2016\)](#); [Wager and Athey \(2018\)](#). Estimates conditional average treatment effects (CATE) $\tau(x) = E[Y(1) - Y(0) | X = x]$ with 95% confidence intervals. First implementation in a Rust ML framework.

Oblique Forest (SPORF). Sparse random projections following [Tomita et al. \(2020\)](#), enabling oblique decision boundaries.

Streaming. Hoeffding Trees for online learning on data streams, using the Hoeffding bound for statistically sound split decisions.

6. Summary and discussion

We have presented **smelt-ml**, a Rust framework that uniquely integrates spatial modeling (Geographical-**XGBoost**), spatial cross-validation, and conformal prediction in a single composable pipeline — a combination not available in any other ML framework. On the performance side, our gradient boosting implementations achieve competitive speed with the

official C++ libraries at research-typical scales ($n \leq 10,000$) while consuming an order of magnitude less memory. The performance advantage at small scales stems from Rust’s ownership model, which guarantees no aliasing (`&mut` is unique) and enables LLVM to apply more aggressive auto-vectorization than is possible with C++, where pointer aliasing must be assumed unless explicitly annotated. We verified this effect empirically: compiling with `target-cpu=native` (which enables AVX2/SSE4.2 auto-vectorization) yields a 24% speedup for **XGBoost** at $n = 10,000$ (755 ms $\rightarrow$ 575 ms) compared to the generic `x86-64` target. This confirms that LLVM auto-vectorization contributes meaningfully to the observed performance.

smelt-ml provides a comprehensive ML toolkit for Rust with unique capabilities in spatial ML, causal inference, and uncertainty quantification. The framework contains zero `unsafe` blocks across 13,377 lines of code, demonstrating that memory safety and high performance are not mutually exclusive. The test suite comprises 320 integration tests covering all learners, preprocessing, resampling, tuning, serialization, and edge cases (empty datasets, single-sample inputs, NaN features, multiclass targets, convergence checks against **scikit-learn**).

Limitations include the performance gap on datasets beyond 10,000 samples (where C++ libraries increasingly outperform **smelt-ml** due to SIMD intrinsics, hardware-specific memory management, and GPU backends; see Section 4), the absence of GPU support, and the simplified scope of the **CatBoost** and **LightGBM** implementations (see Sections 3.4–3.5).

Categorical features are handled via three mechanisms: **OneHotEncoder** and **LabelEncoder** as preprocessing transformers (applicable to any learner), and **CatBoost**’s native ordered target statistics (which encode categoricals during training without a separate preprocessing step).

Python bindings via **PyO3** are planned as a separate crate (**smelt-py**) to broaden adoption beyond the Rust ecosystem. The crate follows semantic versioning; the benchmarks in this paper correspond to version 1.3.0.

The software is available at <https://crates.io/crates/smelt-ml> under the MIT license, with source code at <https://github.com/franciscoparrao/smelt>.

Computational details

The results in this paper were obtained using Rust 1.89 with the **ndarray** 0.16, **rayon** 1.10, and **serde** 1.0 crates. Official C++ libraries were accessed via Python 3.12 with **xgboost** 3.1, **lightgbm** 4.6, and **catboost** 1.2. **scikit-learn** 1.8 was used for accuracy validation. Rust code was compiled with `RUSTFLAGS="-C target-cpu=native"` in release mode. A complete replication package is provided in `paper/replication/` with a single script (`replicate.sh`) that reproduces all benchmark tables and figures.

Acknowledgments

The author thanks the anonymous reviewers for their constructive feedback.

References

Athey S, Imbens G (2016). “Recursive Partitioning for Heterogeneous Causal Effects.” *Pro-*

- ceedings of the National Academy of Sciences*, **113**(27), 7353–7360. doi:[10.1073/pnas.1510489113](https://doi.org/10.1073/pnas.1510489113).
- Chen T, Guestrin C (2016). “**XGBoost**: A Scalable Tree Boosting System.” In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. doi:[10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- Grekousis G (2024). **geoxgboost**: Geographically Weighted **XGBoost**. Python package on PyPI, URL <https://pypi.org/project/geoxgboost/>.
- Grekousis G (2025). “Geographical-**XGBoost**: A New Ensemble Model for Spatially Local Regression Based on Gradient-Boosted Trees.” *Journal of Geographical Systems*, **27**(2), 169–195. doi:[10.1007/s10109-025-00465-4](https://doi.org/10.1007/s10109-025-00465-4).
- Grinsztajn L, Oyallon E, Varoquaux G (2022). “Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?” In *Advances in Neural Information Processing Systems* 35.
- Ishwaran H, Kogalur UB, Blackstone EH, Lauer MS (2008). “Random Survival Forests.” *Annals of Applied Statistics*, **2**(3), 841–860. doi:[10.1214/08-A0AS169](https://doi.org/10.1214/08-A0AS169).
- Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, Ye Q, Liu TY (2017). “**LightGBM**: A Highly Efficient Gradient Boosting Decision Tree.” In *Advances in Neural Information Processing Systems* 30.
- Lang M, Binder M, Richter J, Schratz P, Pfisterer F, Coors S, Au Q, Casalicchio G, Kotthoff L, Bischl B (2019). “**mlr3**: A Modern Object-Oriented Machine Learning Framework in R.” *Journal of Open Source Software*, **4**(44), 1903. doi:[10.21105/joss.01903](https://doi.org/10.21105/joss.01903).
- Matsakis ND, Klock FS (2014). “The Rust Language.” *Ada Letters*, **34**(3), 103–104. doi:[10.1145/2692956.2663188](https://doi.org/10.1145/2692956.2663188).
- Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay É (2011). “**Scikit-learn**: Machine Learning in Python.” *Journal of Machine Learning Research*, **12**, 2825–2830.
- Prokhorenkova L, Gusev G, Vorobev A, Dorogush AV, Gulin A (2018). “**CatBoost**: Unbiased Boosting with Categorical Features.” In *Advances in Neural Information Processing Systems* 31.
- Roberts DR, Bahn V, Ciuti S, Boyce MS, Elith J, Guillera-Aroita G, Hauenstein S, Lahoz-Monfort JJ, Schröder B, Thuiller W, Warton JA, Wintle BA, Hartig F, Dormann CF (2017). “Cross-Validation Strategies for Data with Temporal, Spatial, Hierarchical, or Phylogenetic Structure.” *Ecography*, **40**(8), 913–929. doi:[10.1111/ecog.02881](https://doi.org/10.1111/ecog.02881).
- Romano Y, Patterson E, Candès E (2019). “Conformalized Quantile Regression.” In *Advances in Neural Information Processing Systems* 32.
- rust-ml (2024). **linfa**: A Rust Machine Learning Framework. Version 0.7, URL <https://github.com/rust-ml/linfa>.

Rust Team (2024). *The Rust Programming Language*. Version 1.89, URL <https://www.rust-lang.org/>.

Tomita TM, Browne J, Shen C, Chung J, Patsolic JL, Falk B, Priebe CE, Yim J, Burns R, Maggioni M, Vogelstein JT (2020). “Sparse Projection Oblique Randomer Forests.” *Journal of Machine Learning Research*, **21**(104), 1–39.

Wager S, Athey S (2018). “Estimation and Inference of Heterogeneous Treatment Effects Using Random Forests.” *Journal of the American Statistical Association*, **113**(523), 1228–1242. doi:10.1080/01621459.2017.1319839.

Affiliation:

Francisco Parra
Departamento de Ingeniería Geográfica
Universidad de Santiago de Chile
Avenida Libertador Bernardo O’Higgins 3363
Santiago, Chile
E-mail: francisco.parra.o@usach.cl